

MATH 551: NUMERICAL ORDINARY DIFFERENTIAL EQUATIONS

RUIZHE QIAN

CONTENTS

1. August 24, 2026	2
1.1. Ordinary differential equations	2
1.2. Examples	2
1.3. Solutions and the Lipschitz condition	4
2. August 26, 2026	5
2.1. Stability of solutions	5
2.2. The solution map	7
2.3. One-step methods	8
3. August 28, 2026	9
3.1. Methods obtained from quadrature	10
3.2. Local consistency error	11
3.3. Global error and convergence	12
4. August 31, 2026	12
4.1. From local consistency to global convergence	12
4.2. Runge–Kutta methods	14
5. September 2, 2026	15
5.1. Butcher tableaux	15
5.2. The internal consistency relation	16
5.3. One-step error and order	17
5.4. Examples	18
5.5. Explicit three-stage methods	19
6. September 4, 2026	19
6.1. Runge–Kutta methods and order conditions	19
6.2. Explicit three-stage methods of order three	21
6.3. Explicit four-stage methods of order four	22

6.4. Embedded pairs	23
6.5. The stability function	24

1. AUGUST 24, 2026

1.1. **Ordinary differential equations.** An ODE has the form

$$x'(t) = f(t, x(t)).$$

Given an initial condition $x(t_0) = x_0$, the corresponding *initial-value problem* (IVP) is to find $x(t)$ for $t \geq t_0$ such that

$$(1) \quad x'(t) = f(t, x(t)), \quad x(t_0) = x_0.$$

Topics arising in the numerical solution of differential equations include

- non-stiff problems and stiff solvers;
- geometric solvers;
- parameter estimation and Bayesian approaches, where the vector field may depend on a model parameter θ , as in $f(t, x, \theta)$;
- boundary-value problems; and
- stochastic differential equations (SDEs).

We generally consider vector-valued solutions and vector fields,

$$x: \mathbb{R} \longrightarrow \mathbb{R}^n, \quad f: \mathbb{R} \times \mathbb{R}^n \longrightarrow \mathbb{R}^n,$$

which can be written componentwise as

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad f(t, x) = \begin{bmatrix} f_1(t, x_1, \dots, x_n) \\ \vdots \\ f_n(t, x_1, \dots, x_n) \end{bmatrix}.$$

1.2. **Examples.**

Example 1. *The scalar equation*

$$x' = x$$

is an ODE with $n = 1$.

Example 2 (Damped oscillator). *Consider the second-order equation*

$$x'' = -\omega^2 x - x'.$$

Setting $x_1 = x$ and $x_2 = x'$ converts it into the first-order system

$$\begin{cases} x_1' = x_2, \\ x_2' = -\omega^2 x_1 - x_2. \end{cases}$$

Thus, with $\mathbf{x} = (x_1, x_2)^\top$, we have $\mathbf{x}' = \mathbf{f}(x_1, x_2)$.

Example 3 (Chemical kinetics). *A three-species kinetic model is*

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \begin{cases} x_1' = -k_1 x_1 + k_2 x_2 x_3, \\ x_2' = k_1 x_1 - k_2 x_2 x_3 - k_3 x_2^2, \\ x_3' = k_3 x_2^2, \end{cases}$$

where $k_1, k_2, k_3 > 0$ are reaction rates. The reaction rates may occur on very different scales, which can lead to stiffness.

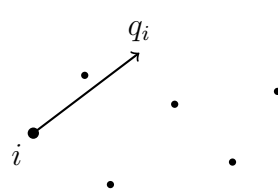
Example 4 (Molecular model). *Let q_i be the position of particle i , let m_i be its mass, and let $V(q_1, \dots, q_N)$ be the potential energy. Newton's equation is*

$$m_i q_i'' = -\frac{\partial V(q_1, \dots, q_N)}{\partial q_i}.$$

Introducing the momentum $p_i = m_i q_i'$ gives

$$\begin{cases} q_i' = \frac{p_i}{m_i}, \\ p_i' = -\frac{\partial V}{\partial q_i}. \end{cases}$$

The positions and momenta can therefore be collected into the state vector

$$\mathbf{x} = \begin{bmatrix} q_1 \\ p_1 \\ q_2 \\ p_2 \\ \vdots \\ q_N \\ p_N \end{bmatrix}.$$


1.3. Solutions and the Lipschitz condition. Consider the IVP (1) on an interval containing t_0 .

Definition 1. A function $x(t)$ is a solution of the IVP if it satisfies the integral equation

$$x(t) = x_0 + \int_{t_0}^t f(s, x(s)) ds.$$

Definition 2 (Lipschitz condition). The vector field f is Lipschitz continuous in x on a domain D if there is a constant $L \geq 0$ such that

$$\|f(t, x) - f(t, y)\| \leq L\|x - y\|$$

for all $x, y \in D$ and all relevant t .

Under the usual continuity assumption in t , the Lipschitz condition in x implies that the solution of the IVP exists and is unique on an appropriate interval.

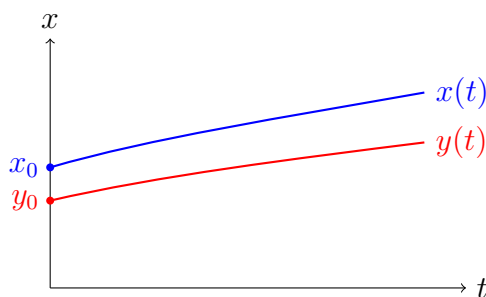
The same condition also controls the sensitivity to the initial data. Let $x(t)$ and $y(t)$ be solutions with initial values x_0 and y_0 , respectively.

Their integral equations give

$$\begin{aligned}\|x(t) - y(t)\| &\leq \|x_0 - y_0\| + \int_{t_0}^t \|f(s, x(s)) - f(s, y(s))\| ds \\ &\leq \|x_0 - y_0\| + L \int_{t_0}^t \|x(s) - y(s)\| ds.\end{aligned}$$

By Grönwall's inequality,

$$(2) \quad \|x(t) - y(t)\| \leq \|x_0 - y_0\| e^{L(t-t_0)}.$$



Thus nearby initial values produce solutions whose separation is bounded by (2).

2. AUGUST 26, 2026

The main topics are properties of solutions, time discretization, and consistency error. We continue to consider the IVP

$$(3) \quad x'(t) = f(t, x(t)), \quad x(t_0) = x_0,$$

where $x: \mathbb{R} \rightarrow \mathbb{R}^n$.

2.1. Stability of solutions. The integral formulation of (3) is

$$x(t) = x_0 + \int_{t_0}^t f(s, x(s)) ds.$$

If $y(t)$ is another solution with initial value y_0 , then

$$y(t) = y_0 + \int_{t_0}^t f(s, y(s)) ds,$$

and hence

$$y(t) - x(t) = y_0 - x_0 + \int_{t_0}^t (f(s, y(s)) - f(s, x(s))) ds.$$

Suppose f is Lipschitz continuous in x with Lipschitz constant L , so that

$$\|f(t, y) - f(t, x)\| \leq L\|y - x\|.$$

Taking norms gives

$$(4) \quad \|y(t) - x(t)\| \leq \|y_0 - x_0\| + \int_{t_0}^t L\|y(s) - x(s)\| ds.$$

The following inequality controls this accumulation of error.

Lemma 1 (Grönwall's inequality). *Let $u, v \in C([t_0, t_1], \mathbb{R})$ be nonnegative functions and let $\rho \geq 0$. If*

$$(5) \quad u(t) \leq \rho + \int_{t_0}^t v(s)u(s) ds, \quad t \in [t_0, t_1],$$

then

$$(6) \quad u(t) \leq \rho \exp \left(\int_{t_0}^t v(s) ds \right), \quad t \in [t_0, t_1].$$

In particular, $u \equiv 0$ when $\rho = 0$.

Proof. First suppose that $\rho > 0$ and define

$$U(t) = \rho + \int_{t_0}^t v(s)u(s) ds.$$

Then $U \in C^1([t_0, t_1], \mathbb{R})$, and

$$u(t) \leq U(t), \quad U(t) \geq \rho > 0.$$

Since u and v are nonnegative,

$$U'(t) = v(t)u(t) \leq v(t)U(t).$$

Thus the logarithmic derivative of U satisfies

$$(\log U(t))' = \frac{U'(t)}{U(t)} \leq v(t).$$

Integrating from t_0 to t yields

$$\log U(t) - \log U(t_0) \leq \int_{t_0}^t v(s) ds.$$

Because $U(t_0) = \rho$ and $u(t) \leq U(t)$, applying the exponential function proves (6) when $\rho > 0$.

Now suppose that $\rho = 0$. For any $\varepsilon > 0$, (5) implies

$$u(t) \leq \varepsilon + \int_{t_0}^t v(s)u(s) ds.$$

The first part of the proof, with $\rho = \varepsilon$, gives

$$0 \leq u(t) \leq \varepsilon \exp \left(\int_{t_0}^t v(s) ds \right).$$

Letting $\varepsilon \downarrow 0$ shows that $u(t) = 0$ for every $t \in [t_0, t_1]$. □

Apply Lemma 1 to (4) with

$$u(t) = \|y(t) - x(t)\|, \quad v(t) = L, \quad \rho = \|y_0 - x_0\|.$$

We obtain

$$(7) \quad \|y(t) - x(t)\| \leq \|y_0 - x_0\| e^{L(t-t_0)}.$$

This estimate can be pessimistic, but it establishes continuous dependence on the initial data.

2.2. The solution map. The *solution map*, or *flow map*, is denoted by

$$x(t) = \Phi^{t,t_0}(x_0).$$

For example, consider the scalar logistic equation

$$x' = x(1 - x).$$

For $x_0 \neq 0$, its solution is

$$\Phi^{t,t_0}(x_0) = \frac{1}{1 + \frac{1-x_0}{x_0} e^{-(t-t_0)}}.$$

When $x_0 = 0$, the solution is $x(t) \equiv 0$.

The solution map has the identity and composition properties

$$\begin{aligned}\Phi^{t_0, t_0}(x_0) &= x_0, & \Phi^{t_0, t_0} &= I, \\ \Phi^{t_2, t_1}(\Phi^{t_1, t_0}(x_0)) &= \Phi^{t_2, t_0}(x_0).\end{aligned}$$

The second property is the basis of time marching.

Differentiating the solution map with respect to time gives

$$\begin{aligned}\frac{d}{dt}\Phi^{t, t_0}(x_0) &= f(t, x(t)), \\ \frac{d^2}{dt^2}\Phi^{t, t_0}(x_0) &= f_t(t, x(t)) + f_x(t, x(t))x'(t) \\ &= (f_t + f_x f)(t, x(t)),\end{aligned}$$

where f_x is the Jacobian matrix of f with respect to x . Higher time derivatives can be found by repeated differentiation, for example with automatic differentiation; they can also be viewed through the Koopman operator.

To measure sensitivity with respect to the initial value, define the Wronskian matrix

$$W(t) = \frac{\partial}{\partial x_0}\Phi^{t, t_0}(x_0) = \frac{\partial x(t)}{\partial x_0}.$$

Differentiating with respect to t gives

$$\begin{aligned}W'(t) &= \frac{\partial}{\partial x_0}f(t, x(t)) \\ &= f_x(t, x(t))\frac{\partial x(t)}{\partial x_0}.\end{aligned}$$

Thus, if

$$A(t) = f_x(t, x(t)),$$

then W satisfies the *variational equation*

$$(8) \quad W'(t) = A(t)W(t), \quad W(t_0) = I.$$

2.3. One-step methods. Choose a time grid

$$t_0 < t_1 < \cdots < t_N = t, \quad \tau_j = t_{j+1} - t_j,$$

where the step sizes τ_j are small. Repeated use of the composition property gives

$$\Phi^{t_N, t_0} = \Phi^{t_N, t_{N-1}} \circ \dots \circ \Phi^{t_2, t_1} \circ \Phi^{t_1, t_0}.$$

It is therefore enough to approximate a single short-time solution map. Writing $t_j = t$ and $t_{j+1} = t + \tau$, Taylor expansion gives

$$(9) \quad \Phi^{t+\tau, t}(x) = x + \tau f(t, x) + \frac{\tau^2}{2} (f_t + f_x f)(t, x) + O(\tau^3).$$

Dropping the terms of order τ^2 and higher defines the Euler one-step map

$$\Psi^{t+\tau, t}(x) = x + \tau f(t, x).$$

Consequently, Euler's method is

$$(10) \quad x_{j+1} = x_j + \tau_j f(t_j, x_j).$$

Its one-step consistency error follows from (9):

$$\Phi^{t+\tau, t}(x) - \Psi^{t+\tau, t}(x) = \frac{\tau^2}{2} (f_t + f_x f)(t, x) + O(\tau^3) = O(\tau^2).$$

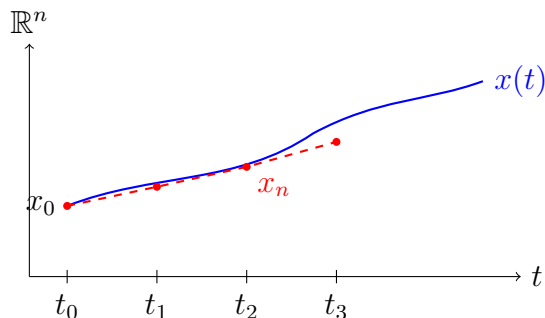
3. AUGUST 28, 2026

We continue the study of one-step methods, focusing on consistency, examples, and convergence. Let

$$t_0 < t_1 < \dots < t_N, \quad \tau_n = t_{n+1} - t_n,$$

where t_n is a time level and τ_n is the corresponding step size. We use $x(t_n)$ for the exact solution and x_n for its numerical approximation:

$$x(t_n) \approx x_n.$$



A one-step method advances the numerical solution by a map

$$(11) \quad x_{n+1} = \Psi^{t_{n+1}, t_n}(x_n).$$

Thus the exact and numerical solutions at t_n can be written as

$$\begin{aligned} x(t_n) &= \Phi^{t_n, t_{n-1}} \circ \dots \circ \Phi^{t_2, t_1} \circ \Phi^{t_1, t_0}(x_0), \\ x_n &= \Psi^{t_n, t_{n-1}} \circ \dots \circ \Psi^{t_2, t_1} \circ \Psi^{t_1, t_0}(x_0). \end{aligned}$$

3.1. Methods obtained from quadrature. The integral form of the ODE over one step is

$$(12) \quad x(t_{n+1}) = x(t_n) + \int_{t_n}^{t_{n+1}} f(s, x(s)) ds.$$

Different quadrature approximations of this integral produce different one-step methods.

3.1.1. Forward Euler method. Using the left endpoint in (12) gives

$$x(t_{n+1}) \approx x(t_n) + \tau_n f(t_n, x(t_n)).$$

Replacing the exact solution values by numerical approximations yields

$$(13) \quad x_{n+1} = x_n + \tau_n f(t_n, x_n) = \Psi_{\text{FE}}^{t_{n+1}, t_n}(x_n).$$

3.1.2. Backward Euler method. Using the right endpoint instead gives

$$x(t_{n+1}) \approx x(t_n) + \tau_n f(t_{n+1}, x(t_{n+1})).$$

The resulting method is

$$(14) \quad x_{n+1} = x_n + \tau_n f(t_{n+1}, x_{n+1}) = \Psi_{\text{BE}}^{t_{n+1}, t_n}(x_n).$$

Because x_{n+1} occurs on both sides, backward Euler requires the solution of a fixed-point problem at each step.

3.1.3. A predictor–corrector method. The trapezoidal rule suggests

$$x(t_{n+1}) \approx x(t_n) + \frac{\tau_n}{2} (f(t_n, x(t_n)) + f(t_{n+1}, x(t_{n+1}))).$$

We can first predict the right endpoint with forward Euler and then use that prediction in the trapezoidal formula:

$$(15) \quad x_{n+1}^* = x_n + \tau_n f(t_n, x_n),$$

$$(16) \quad x_{n+1} = x_n + \frac{\tau_n}{2} (f(t_n, x_n) + f(t_{n+1}, x_{n+1}^*)).$$

This explicit two-stage scheme is also called Heun's method. Together, the two stages define a one-step map

$$x_{n+1} = \Psi_H^{t_{n+1}, t_n}(x_n).$$

3.2. Local consistency error. For a numerical one-step map Ψ , define its *local error*, or *consistency error*, at (t_n, x) by

$$(17) \quad \mathcal{E}_n(x) = \Phi^{t_{n+1}, t_n}(x) - \Psi^{t_{n+1}, t_n}(x).$$

Here the exact and numerical steps start from the same value x at time t_n .

Example 5 (Forward Euler). *The Taylor expansion from (9) gives*

$$\begin{aligned} \mathcal{E}_n^{\text{FE}}(x) &= \Phi^{t_n + \tau_n, t_n}(x) - (x + \tau_n f(t_n, x)) \\ &= \frac{\tau_n^2}{2} (f_t + f_x f)(t_n, x) + O(\tau_n^3) \\ &= O(\tau_n^2). \end{aligned}$$

Example 6 (Backward Euler). *Let*

$$y(\tau) = \Psi_{\text{BE}}^{t_n + \tau, t_n}(x).$$

The defining fixed-point equation is

$$y(\tau) = x + \tau f(t_n + \tau, y(\tau)).$$

Differentiating this equation and evaluating at $\tau = 0$ gives

$$y'(0) = f(t_n, x), \quad y''(0) = 2(f_t + f_x f)(t_n, x).$$

Therefore,

$$\Psi_{\text{BE}}^{t_n + \tau_n, t_n}(x) = x + \tau_n f(t_n, x) + \tau_n^2 (f_t + f_x f)(t_n, x) + O(\tau_n^3).$$

Comparing this with the exact flow expansion yields

$$\begin{aligned}\mathcal{E}_n^{\text{BE}}(x) &= -\frac{\tau_n^2}{2}(f_t + f_x f)(t_n, x) + O(\tau_n^3) \\ &= O(\tau_n^2).\end{aligned}$$

3.3. Global error and convergence. The *global error* at t_n is

$$(18) \quad e_n = x_n - x(t_n).$$

The local errors accumulate as the method marches forward in time. On a fixed time interval there are approximately $1/\tau$ steps of size τ , so one generally loses one power of τ when passing from local to global error:

$$\mathcal{E}_n = O(\tau^{p+1}) \implies e_n = O(\tau^p).$$

More generally, with

$$h = \max_{0 \leq n < N} \tau_n,$$

a stable one-step method with local error $O(h^{p+1})$ has global error $O(h^p)$ under the usual smoothness assumptions. Thus both forward and backward Euler have local error of order two and global order one.

4. AUGUST 31, 2026

We continue the analysis of one-step methods. The main goals are to pass from local consistency to global convergence and to introduce Runge–Kutta methods. Consider the IVP

$$x'(t) = f(t, x(t)), \quad x(t_0) = x_0 \in \mathbb{R}^n,$$

on a grid

$$t_0 < t_1 < \cdots < t_N = T, \quad \tau_j = t_{j+1} - t_j, \quad h = \max_{0 \leq j < N} \tau_j.$$

4.1. From local consistency to global convergence. Recall that Φ^{t_{j+1}, t_j} denotes the exact flow over one step, while Ψ^{t_{j+1}, t_j} denotes the numerical one-step map. For a starting value x , define the local

consistency error by

$$(19) \quad \xi_j(x) = \Phi^{t_{j+1}, t_j}(x) - \Psi^{t_{j+1}, t_j}(x).$$

Definition 3 (Consistency order). *A one-step method has consistency order $p > 0$ if, uniformly for the relevant starting values,*

$$\|\xi_j(x)\| \leq C\tau_j^{p+1}$$

as $h \rightarrow 0$. The order p need not be an integer when the data have only Hölder regularity.

Forward and backward Euler have consistency order one. The exact and numerical values at the final time can be written as compositions:

$$\begin{aligned} x(T) &= \Phi^{t_N, t_{N-1}} \circ \dots \circ \Phi^{t_2, t_1} \circ \Phi^{t_1, t_0}(x_0), \\ x_N &= \Psi^{t_N, t_{N-1}} \circ \dots \circ \Psi^{t_2, t_1} \circ \Psi^{t_1, t_0}(x_0). \end{aligned}$$

Thus the local errors are propagated through all the subsequent steps.

Let

$$e_j = x_j - x(t_j)$$

be the global error. For brevity, set $\Psi_j = \Psi^{t_{j+1}, t_j}$ and $\Phi_j = \Phi^{t_{j+1}, t_j}$. Then

$$\begin{aligned} e_{j+1} &= \Psi_j(x_j) - \Phi_j(x(t_j)) \\ (20) \quad &= \Psi_j(x_j) - \Psi_j(x(t_j)) - \xi_j(x(t_j)). \end{aligned}$$

Suppose the one-step map satisfies the stability estimate

$$(21) \quad \|\Psi_j(y) - \Psi_j(z)\| \leq e^{L\tau_j} \|y - z\|.$$

Combining the consistency and stability estimates in (20) gives

$$(22) \quad \|e_{j+1}\| \leq e^{L\tau_j} \|e_j\| + C\tau_j^{p+1}.$$

Since $e_0 = 0$, iteration yields

$$\|e_N\| \leq C \sum_{j=0}^{N-1} \tau_j^{p+1} e^{L(T-t_{j+1})}$$

$$\leq Ch^p \sum_{j=0}^{N-1} \tau_j e^{L(T-t_{j+1})}.$$

For $L > 0$, the sum is bounded by a geometric integral, and hence

$$(23) \quad \|e_N\| \leq \frac{C}{L} (e^{L(T-t_0)} - 1) h^p.$$

When $L = 0$, the corresponding bound is

$$\|e_N\| \leq C(T - t_0) h^p.$$

In either case, $h \rightarrow 0$ implies $e_N \rightarrow 0$. Thus stability converts a local error of order $p + 1$ into a global error of order p .

Remark 1 (Contractive problems). In the favorable case

$$\|\Psi_j(y) - \Psi_j(z)\| \leq \|y - z\|,$$

no exponential amplification occurs. Direct summation gives

$$\|e_N\| \leq C \sum_{j=0}^{N-1} \tau_j^{p+1} \leq C(T - t_0) h^p.$$

Such contractivity is especially relevant for dissipative ODEs and numerical methods that preserve their dissipative structure.

4.2. Runge–Kutta methods. Consider an explicit method with s stages and coefficients a_{ij} , b_i , and c_i . Starting from x at time t with step size τ , its stages are computed successively from

$$(24) \quad k_i = f \left(t + c_i \tau, x + \tau \sum_{j=1}^{i-1} a_{ij} k_j \right), \quad i = 1, \dots, s,$$

and its one-step map is

$$(25) \quad \Psi^{t+\tau, t}(x) = x + \tau \sum_{i=1}^s b_i k_i.$$

Example 7 (Forward Euler). Take $s = 1$, $c_1 = 0$, $a_{11} = 0$, and $b_1 = 1$. Then

$$k_1 = f(t, x), \quad \Psi^{t+\tau, t}(x) = x + \tau k_1,$$

which is forward Euler, a first-order method.

Example 8 (Explicit trapezoidal method). *Take $s = 2$ and*

$$A = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \end{bmatrix}, \quad c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The stages and update are

$$\begin{aligned} k_1 &= f(t, x), \\ k_2 &= f(t + \tau, x + \tau k_1), \\ \Psi^{t+\tau, t}(x) &= x + \frac{\tau}{2}(k_1 + k_2). \end{aligned}$$

This is the predictor–corrector method from the previous lecture, also called Heun’s method. It has order two.

5. SEPTEMBER 2, 2026

We develop the coefficient representation of Runge–Kutta methods and derive their order conditions. Throughout, we retain the notation of the previous lectures: $\tau_n = t_{n+1} - t_n$, x_n is the numerical approximation at t_n , Φ is the exact flow, and Ψ is the numerical one-step map.

5.1. Butcher tableaux. An s -stage Runge–Kutta method has the form

$$(26) \quad k_i = f \left(t_n + c_i \tau_n, x_n + \tau_n \sum_{j=1}^s a_{ij} k_j \right), \quad i = 1, \dots, s,$$

$$(27) \quad x_{n+1} = x_n + \tau_n \sum_{i=1}^s b_i k_i.$$

The coefficients are collected into

$$A = (a_{ij})_{i,j=1}^s, \quad b = \begin{bmatrix} b_1 & \cdots & b_s \end{bmatrix}^\top, \quad c = \begin{bmatrix} c_1 & \cdots & c_s \end{bmatrix}^\top.$$

They are conventionally displayed in a *Butcher tableau*:

$$\begin{array}{c|cccc}
 c_1 & a_{11} & a_{12} & \cdots & a_{1s} \\
 c_2 & a_{21} & a_{22} & \cdots & a_{2s} \\
 \vdots & \vdots & \vdots & \ddots & \vdots \\
 c_s & a_{s1} & a_{s2} & \cdots & a_{ss} \\
 \hline
 & b_1 & b_2 & \cdots & b_s
 \end{array}$$

The values $t_n + c_i \tau_n$ are the intermediate stage times between t_n and t_{n+1} . If A is strictly lower triangular, then each stage depends only on previously computed stages. The resulting method is an *explicit Runge–Kutta method*.

5.2. The internal consistency relation. Any nonautonomous ODE can be converted into an autonomous system by augmenting its state. Set

$$z(t) = \begin{bmatrix} t \\ x(t) \end{bmatrix}, \quad z'(t) = \begin{bmatrix} 1 \\ f(t, x(t)) \end{bmatrix} = F(z(t)).$$

Apply a Runge–Kutta method to this autonomous system. Because the time component of every stage slope is one, the time component of stage i is

$$t_n + \tau_n \sum_{j=1}^s a_{ij}.$$

For this to agree with the prescribed stage time $t_n + c_i \tau_n$ in (26), we require

$$c_i = \sum_{j=1}^s a_{ij}, \quad i = 1, \dots, s.$$

Thus the *internal consistency relation* is

$$(28) \quad c = A\mathbf{1}, \quad \mathbf{1} = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}^T.$$

Under this relation, applying the Runge–Kutta scheme directly to the nonautonomous equation is equivalent to applying it to the augmented autonomous system.

5.3. One-step error and order. For an exact starting value x at time t , define the one-step defect by

$$(29) \quad \mathcal{E}(x, t, \tau) = \Phi^{t+\tau, t}(x) - \Psi^{t+\tau, t}(x).$$

A Runge–Kutta method has order p if

$$\mathcal{E}(x, t, \tau) = O(\tau^{p+1}) \quad \text{as } \tau \rightarrow 0.$$

Assuming (28), it suffices to derive the conditions for an autonomous equation

$$x' = f(x).$$

We write its exact and Runge–Kutta flows as Φ^τ and Ψ^τ , respectively.

Let $f' = Df(x)$ and $f'' = D^2f(x)$. Differentiation along an exact solution gives

$$\begin{aligned} x' &= f, \\ x'' &= f'f, \\ x''' &= f''(f, f) + f'f'f. \end{aligned}$$

Consequently,

$$(30) \quad \Phi^\tau(x) = x + \tau f + \frac{\tau^2}{2} f'f + \frac{\tau^3}{6} (f''(f, f) + f'f'f) + O(\tau^4).$$

For the numerical method, set

$$\Delta_i = \tau \sum_{j=1}^s a_{ij} k_j, \quad k_i = f(x + \Delta_i).$$

Since $k_j = f + O(\tau)$ and $c = A\mathbf{1}$,

$$\Delta_i = \tau c_i f + O(\tau^2).$$

A more detailed Taylor expansion gives

$$k_i = f + \tau c_i f'f + \tau^2 \left(\sum_{j=1}^s a_{ij} c_j f'f'f + \frac{c_i^2}{2} f''(f, f) \right) + O(\tau^3).$$

Substitution into $\Psi^\tau(x) = x + \tau \sum_i b_i k_i$ yields

$$\Psi^\tau(x) = x + \tau(b^\top \mathbf{1})f + \tau^2(b^\top c)f'f$$

$$(31) \quad + \tau^3 \left((b^\top A c) f' f' f + \frac{1}{2} (b^\top c^{\circ 2}) f''(f, f) \right) + O(\tau^4),$$

where $c^{\circ 2} = (c_1^2, \dots, c_s^2)^\top$.

Comparing (30) and (31) gives the order conditions through order three:

$$(32) \quad b^\top \mathbf{1} = 1, \quad (\text{order one}),$$

$$(33) \quad b^\top c = \frac{1}{2}, \quad (\text{additional condition for order two}),$$

$$(34) \quad b^\top c^{\circ 2} = \frac{1}{3}, \quad (\text{additional condition for order three}),$$

$$(35) \quad b^\top A c = \frac{1}{6}, \quad (\text{additional condition for order three}).$$

Therefore (32) gives a first-order method; the first two conditions give a second-order method; and all four give a third-order method. A fourth-order method must satisfy eight conditions in total. For nonautonomous equations, the relation $c = A\mathbf{1}$ is also needed for these autonomous order conditions to apply directly.

5.4. Examples.

Example 9 (Forward Euler). *Forward Euler has the tableau*

$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}.$$

Thus $s = 1$, $A = 0$, $b = 1$, and $c = 0$. It satisfies $b^\top \mathbf{1} = 1$, but not the second-order condition, so it is first order.

Example 10 (Explicit trapezoidal method). *The explicit trapezoidal method, or Heun's method, is*

$$\begin{aligned} k_1 &= f(t_n, x_n), \\ k_2 &= f(t_n + \tau_n, x_n + \tau_n k_1), \\ x_{n+1} &= x_n + \frac{\tau_n}{2} (k_1 + k_2). \end{aligned}$$

Its tableau is

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1 & 1 & 0 \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}.$$

Here $c = A\mathbf{1}$, $b^\top \mathbf{1} = 1$, and $b^\top c = 1/2$. However, $b^\top c^{\circ 2} = 1/2 \neq 1/3$, so the method is second order but not third order.

5.5. Explicit three-stage methods. A general explicit three-stage Runge–Kutta method has tableau

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ c_2 & a_{21} & 0 & 0 \\ c_3 & a_{31} & a_{32} & 0 \\ \hline & b_1 & b_2 & b_3 \end{array}.$$

Internal consistency requires

$$c_2 = a_{21}, \quad c_3 = a_{31} + a_{32}.$$

Before imposing the order conditions, the six free coefficients are

$$b_1, b_2, b_3, a_{21}, a_{31}, a_{32}.$$

The four conditions (32)–(35) enforce third order, so one generically expects a two-parameter family of explicit three-stage schemes of order three.

6. SEPTEMBER 4, 2026

We continue the study of Runge–Kutta methods, first constructing families of explicit methods and then introducing embedded methods and stability functions. We use the notation established in the previous lectures: $\tau_n = t_{n+1} - t_n$, x_n is the numerical solution, and Φ and Ψ denote the exact and numerical flows.

6.1. Runge–Kutta methods and order conditions. Recall that an s -stage Runge–Kutta method is

$$k_i = f \left(t_n + c_i \tau_n, x_n + \tau_n \sum_{j=1}^s a_{ij} k_j \right), \quad i = 1, \dots, s,$$

$$x_{n+1} = x_n + \tau_n \sum_{i=1}^s b_i k_i.$$

Here s is the number of stages, $A = (a_{ij})$ contains the stage coefficients, c_i is the i -th stage node, and b_i is the i -th quadrature weight. The internal consistency relation is

$$c = A\mathbf{1}.$$

It makes the method agree on the original nonautonomous equation and its autonomous augmentation. The method is explicit precisely when A is strictly lower triangular.

For the autonomous equation $x' = f(x)$, a method has order p when

$$\|\Phi^\tau(x) - \Psi^\tau(x)\| = O(\tau^{p+1}) \quad \text{as } \tau \rightarrow 0.$$

The first four order conditions, derived in the previous lecture, are

$$\begin{aligned} b^\top \mathbf{1} &= 1, & b^\top c &= \frac{1}{2}, \\ b^\top c^{\circ 2} &= \frac{1}{3}, & b^\top Ac &= \frac{1}{6}. \end{aligned}$$

They are sufficient through order three. To obtain order four, four more conditions are needed:

$$\begin{aligned} b^\top c^{\circ 3} &= \frac{1}{4}, & b^\top (c \circ (Ac)) &= \frac{1}{8}, \\ b^\top A(c^{\circ 2}) &= \frac{1}{12}, & b^\top A^2 c &= \frac{1}{24}, \end{aligned}$$

where $\circ$ denotes componentwise multiplication. Thus one, two, four, and eight conditions must be satisfied for orders one, two, three, and four, respectively.

Forward Euler uses one stage and has order one, while the explicit trapezoidal method uses two stages and has order two. We next continue the construction of higher-order explicit methods.

6.2. Explicit three-stage methods of order three. For an explicit three-stage method, internal consistency gives the tableau

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ c_2 & a_{21} & 0 & 0 \\ c_3 & a_{31} & a_{32} & 0 \\ \hline & b_1 & b_2 & b_3 \end{array}, \quad c_2 = a_{21}, \quad c_3 = a_{31} + a_{32}.$$

There are six free coefficients and four order conditions, so the third-order methods generically form a two-parameter family. Let

$$c_2 = \alpha, \quad c_3 = \beta.$$

Solving the four order conditions gives

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ \alpha & \alpha & 0 & 0 \\ \beta & \frac{\beta(3\alpha(1-\alpha)-\beta)}{\alpha(2-3\alpha)} & \frac{\beta(\beta-\alpha)}{\alpha(2-3\alpha)} & 0 \\ \hline & \frac{6\alpha\beta-3\alpha-3\beta+2}{6\alpha\beta} & \frac{3\beta-2}{6\alpha(\beta-\alpha)} & \frac{2-3\alpha}{6\beta(\beta-\alpha)} \end{array},$$

provided the displayed denominators are nonzero.

A useful subfamily has $\beta = 1$. Its weights are

$$b_1 = \frac{3\alpha-1}{6\alpha}, \quad b_2 = \frac{1}{6\alpha(1-\alpha)}, \quad b_3 = \frac{2-3\alpha}{6(1-\alpha)}.$$

All three weights are nonnegative when $\alpha \in [1/3, 2/3]$, but positivity of the weights alone does not guarantee that one method is better than another.

Example 11 (Kutta's third-order method). *Taking $\alpha = 1/2$ and $\beta = 1$ gives*

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 \\ 1 & -1 & 2 & 0 \\ \hline & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \end{array}.$$

Thus

$$k_1 = f(t_n, x_n),$$

$$\begin{aligned}
k_2 &= f\left(t_n + \frac{\tau_n}{2}, x_n + \frac{\tau_n}{2}k_1\right), \\
k_3 &= f\left(t_n + \tau_n, x_n + \tau_n(-k_1 + 2k_2)\right), \\
x_{n+1} &= x_n + \frac{\tau_n}{6}(k_1 + 4k_2 + k_3).
\end{aligned}$$

6.3. Explicit four-stage methods of order four. Before imposing internal consistency, a general explicit four-stage tableau contains thirteen coefficients: three nontrivial nodes, six strictly lower-triangular entries of A , and four weights. Internal consistency removes three degrees of freedom, and the eight fourth-order conditions generically leave two free parameters.

In the usual nonconfluent family one may take

$$c_1 = 0, \quad c_2 = \alpha, \quad c_3 = \beta, \quad c_4 = 1,$$

with distinct stage nodes. The generic formulas exclude degenerate parameter choices; one convenient nondegeneracy condition is

$$\alpha\beta(\alpha - 1)(\beta - 1)(\alpha - \beta)(2\alpha - 1)(6\alpha\beta - 4\alpha - 4\beta + 3) \neq 0.$$

Example 12 (The three-eighths rule). *Choosing $\alpha = 1/3$ and $\beta = 2/3$ produces the fourth-order three-eighths rule*

$$\begin{array}{c|cccc}
0 & 0 & 0 & 0 & 0 \\
\frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 \\
\frac{2}{3} & -\frac{1}{3} & 1 & 0 & 0 \\
1 & 1 & -1 & 1 & 0 \\
\hline
& \frac{1}{8} & \frac{3}{8} & \frac{3}{8} & \frac{1}{8}
\end{array} .$$

Its stages are

$$\begin{aligned}
k_1 &= f(t_n, x_n), \\
k_2 &= f\left(t_n + \frac{\tau_n}{3}, x_n + \frac{\tau_n}{3}k_1\right), \\
k_3 &= f\left(t_n + \frac{2\tau_n}{3}, x_n + \tau_n\left(-\frac{1}{3}k_1 + k_2\right)\right), \\
k_4 &= f\left(t_n + \tau_n, x_n + \tau_n(k_1 - k_2 + k_3)\right), \\
x_{n+1} &= x_n + \frac{\tau_n}{8}(k_1 + 3k_2 + 3k_3 + k_4).
\end{aligned}$$

Repeated nodes give a separate, *confluent* family. Setting $c_2 = c_3 = 1/2$ and letting $\delta = a_{32} \neq 0$ yields

$$(36) \quad \begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} - \delta & \delta & 0 & 0 \\ 1 & 0 & 1 - \frac{1}{2\delta} & \frac{1}{2\delta} & 0 \\ \hline & \frac{1}{6} & \frac{4\delta - 1}{6\delta} & \frac{1}{6\delta} & \frac{1}{6} \end{array}.$$

Example 13 (Classical RK4). *The choice $\delta = 1/2$ in (36) gives the classical tableau*

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}.$$

Equivalently,

$$\begin{aligned} k_1 &= f(t_n, x_n), \\ k_2 &= f\left(t_n + \frac{\tau_n}{2}, x_n + \frac{\tau_n}{2}k_1\right), \\ k_3 &= f\left(t_n + \frac{\tau_n}{2}, x_n + \frac{\tau_n}{2}k_2\right), \\ k_4 &= f(t_n + \tau_n, x_n + \tau_n k_3), \\ x_{n+1} &= x_n + \frac{\tau_n}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

6.4. Embedded pairs. Adaptive solvers need local error estimates. They obtain them by combining the same stages in two different ways. Given fixed A and c , choose two weight vectors b and $\widehat{b}$ and form

$$\begin{aligned} x_{n+1}^{[p]} &= x_n + \tau_n \sum_{i=1}^s b_i k_i, \\ x_{n+1}^{[p+1]} &= x_n + \tau_n \sum_{i=1}^s \widehat{b}_i k_i. \end{aligned}$$

The two formulas share all evaluations k_i , so

$$\text{err}_n = \|x_{n+1}^{[p+1]} - x_{n+1}^{[p]}\|$$

is an inexpensive local error estimate. If this quantity is well below the requested tolerance, the next step size can be increased. If it is too large, the step size is decreased and the step may be rejected.

MATLAB's `ode45` uses a seven-stage Dormand–Prince embedded pair of orders four and five. The minimal number of stages for an explicit Runge–Kutta method grows with its order; the low-order values discussed in class are

p	1	2	3	4	5	6	7	8
$s_{\min}$	1	2	3	4	6	7	9	11

6.5. The stability function. Apply a Runge–Kutta method to the linear test equation

$$(37) \quad y'(t) = \lambda y(t).$$

With $z = \tau_n \lambda$, one numerical step has the form

$$(38) \quad y_{n+1} = R(z)y_n,$$

where R is the *stability function*. The exact amplification factor is e^z .

For a Runge–Kutta method, the stage equations applied to (37) give

$$(I - zA)k = \lambda y_n \mathbf{1}.$$

Substituting the resulting stages into the update gives

$$(39) \quad R(z) = 1 + zb^T(I - zA)^{-1}\mathbf{1}.$$

If the method has order p , then necessarily

$$(40) \quad R(z) - e^z = O(z^{p+1}) \quad \text{as } z \rightarrow 0.$$

For example, the eight fourth-order conditions imply

$$R(z) = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!} + O(z^5).$$

Condition (40) is necessary for order p , but it only tests a linear ODE and is not sufficient to establish order p for general nonlinear equations.